

БАЗЫ ДАННЫХ / KUBERNETES

Как запускать базы данных в Kubernetes: руководство из 8 шагов

Из этого пошагового руководства вы узнаете, как запускать MySQL, PostgreSQL, MongoDB и другие приложения с отслеживанием состояния в Kubernetes.

25 октября 2024 года, 11:00 [Адетокунбо Иге](#)



Изображение предоставлено Алексом Шупером для Unsplash+.

[Andela](#) спонсировала этот пост.

Несмотря на то, что сегодня почти никто не сомневается в целесообразности использования Kubernetes (K8s) для управления контейнерными приложениями, многие инженеры (в том числе и я) по-прежнему скептически относятся к запуску **баз данных в Kubernetes**. Поскольку базы данных — это, как правило, **приложения с отслеживанием состояния**, им требуется постоянное хранение данных и согласованность, а Kubernetes завоевал свою репутацию благодаря приложениям без отслеживания состояния. Поэтому,

восстановление, а также высокую доступность и аварийное переключение.

В этом руководстве я на примере создания и запуска базы данных MySQL в Kubernetes продемонстрирую, как управлять приложениями с отслеживанием состояния в Kubernetes. Я подробно расскажу о таких ключевых понятиях, как StatefulSets, PersistentVolumes (PV), PersistentVolumeClaims (PVC) и StorageClasses. Я предполагаю, что вы уже имеете представление о **базах данных** и **Kubernetes**.

Прежде чем я начну, важно понять разницу между приложениями без сохранения состояния и приложениями с сохранением состояния. Приложения без сохранения состояния не хранят данные между запросами; каждый запрос обрабатывает данные по отдельности, не заботясь об их совместном использовании. Приложения с сохранением состояния хранят данные между запросами и совместно используют их в рамках сессий или модулей. Такие рабочие нагрузки, как базы данных, требуют постоянного хранения данных.

Ключевые концепции для работы с базами данных в Kubernetes

Для работы с такими базами данных, как **MySQL**, **PostgreSQL** и **MongoDB**, в Kubernetes требуется тщательное планирование постоянного хранилища, стабильных сетевых идентификаторов и стратегий масштабирования. При работе с базой данных в Kubernetes необходимо учитывать следующие детали.

Хранение данных в базе

Для каждого модуля базы данных требуется собственное постоянное хранилище, чтобы обеспечить сохранность данных. Это означает, что даже если модуль будет удален или перезапущен, данные останутся в целости. Каждому модулю базы данных назначается отдельный постоянный и временный пул.

companies scale teams & deliver projects faster via specialized areas: App Engineering, AI, Cloud, Data & Analytics.

Learn More →

HEAR MORE FROM OUR SPONSOR

svo1975pnv1982@gmail.com

Submit

Scaling Databases

When scaling databases, it is very important to ensure data consistency. StatefulSets support running a leader-follower database architecture (primary-secondary), or a primary, read-only replica database, like PostgreSQL or MySQL. The primary database handles updates or writes, while the secondary database replicates or synchronizes, ensuring both consistency and redundancy.

Data Consistency and Backups

It is crucial to have a strategy to ensure data consistency across all database replicas and validate the integrity of the data. Regular backups and disaster recovery plans should be incorporated into your Kubernetes workflows. This must include routine (weekly or monthly) disaster recovery tests to validate the integrity of the database backup.

TRENDING STORIES

1. [The database storage problem is solved. Here's what comes next.](#)
2. [When your data model is the bottleneck: lessons from Medium's feature store](#)
3. [Why MotherDuck refuses to fork DuckDB](#)
4. [Why the need for humans won't disappear in the age of autonomous databases](#)

StatefulSets

A StatefulSet is a Kubernetes resource designed for managing stateful applications such as databases. It ensures that pods possess persistent storage and that data remains intact even when the pods get restarted. Key features of StatefulSets include:

- **Persistent storage:** StatefulSets utilize PVs, which ensure that each pod has dedicated, stable storage that remains intact even after a pod restarts.
- **Stable network identifiers:** Every individual pod in a StatefulSet receives a unique and consistent name, which remains unchanged even after deployment; for example: `mypod-0` , `mypod-1` , `mypod-2` .

Tutorial: Create a Database on Kubernetes

To create a StatefulSet application (such as a database) on Kubernetes, follow this step-by-step guide.

Step 1: Create a StorageClass (if You Don't Have One)

A StorageClass in Kubernetes is similar in concept to a `profile` , as it contains the details of an object. The storage class defines the storage type (either `gp2` or `gp3`) and the parameter for your PV. You can specify a default storage class for dynamic volume provisioning and for any PVC that does not include a specific storage class.

Here is an example of a storage class created for **Amazon** EKS.

Create a new file called `storage-class.yaml` and copy this code into the file.

```
1  apiVersion: storage.k8s.io/v1
2  kind: StorageClass
3  metadata:
4    name: standard
5  provisioner: kubernetes.io/aws-efs # Use the correct provisioner for your cloud p
6  parameters:
7    type: gp3
8  reclaimPolicy: Retain
```

```
kubectl apply -f storage-class.yaml
```

Step 2: Create a PersistentVolume (PV)

```
1  apiVersion: v1
2  kind: PersistentVolume
3  metadata:
4    name: mysql-pv
5  spec:
6    capacity:
7      storage: 10Gi
8    accessModes:
9      - ReadWriteOnce
10   persistentVolumeReclaimPolicy: Retain
11   storageClassName: standard
12   hostPath:
13     path: /mnt/data # Specify a path in the host for storage
```

persistentvolume.yaml hosted with ❤ by GitHub

[view raw](#)

A PV is storage allocated in your Kubernetes cluster. If dynamic provisioning is enabled, Kubernetes will create a PV automatically. Otherwise, you can create one manually.

Step 3: Create a Persistent Volume Claim (PVC)

A PVC serves as an interface between your application and requested storage. A PVC allows your application to request storage from the available PV.

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: mysql-pvc
5  spec:
6    accessModes:
7      - ReadWriteOnce
8    resources:
9      requests:
10       storage: 10Gi
11   storageClassName: standard
```

pvc.yaml hosted with ❤ by GitHub

[view raw](#)

(instance) gets its own unique identifier, persistent storage and stable network identity.

```
1  apiVersion: apps/v1
2  kind: StatefulSet
3  metadata:
4    name: mysql
5  spec:
6    serviceName: "mysql"
7    replicas: 3
8    selector:
9      matchLabels:
10       app: mysql
11   template:
12     metadata:
13       labels:
14         app: mysql
15     spec:
16       containers:
17       - name: mysql
18         image: mysql:5.7
19         ports:
20         - containerPort: 3306
21           name: mysql
22         env:
23         - name: MYSQL_ROOT_PASSWORD
24           value: "your_password"
25         volumeMounts:
26         - name: mysql-storage
27           mountPath: /var/lib/mysql
28   volumeClaimTemplates:
29   - metadata:
30       name: mysql-storage
31     spec:
32       accessModes: [ "ReadWriteOnce" ]
33       resources:
34         requests:
35           storage: 10Gi
36       storageClassName: standard
```

statefulset.yaml hosted with ❤ by GitHub

[view raw](#)

below is named `mysql`. The MySQL pods will be accessible within the cluster by using the name `<pod-name>.mysql` from within any pod in the same Kubernetes namespace and cluster.

```
1 # Headless service
2 apiVersion: v1
3 kind: Service
4 metadata:
5   name: mysql
6   labels:
7     app: mysql
8 spec:
9   ports:
10    - name: mysql
11      port: 3306
12   selector:
13     app: mysql
```

headless_service.yaml hosted with ❤ by GitHub

[view raw](#)

Step 6: Pipe MySQL Logs to Monitoring Tools

Monitoring MySQL is very important in identifying the database performance, bottlenecks, and errors and ensuring database health. The logs from the MySQL StatefulSets can be routed to monitoring tools such as **Datadog**, Grafana, Prometheus and Elasticsearch (the ELK Stack) to get full visibility into the performance and health of the database.

You need to configure MySQL to pipe logs to your monitoring tools. Commonly monitored logs include:

- **Slow query logs** identify slow running logs.
- **Error logs** track errors and warnings.
- **General query logs** track all MySQL queries.

Step 7: Perform Regular Backups and Routine Restore

It is very important to perform regular backups to ensure availability of your Kubernetes workloads and routine restore to validate the integrity of the database.

K8ops is an open source tool designed to safely back up and restore resources on Kubernetes clusters and PVs. It is an excellent solution for ensuring that your

scheduled backups. For more information, check out [Velero's documentation](#).

Step 8: Configure Database Alerts

In a Kubernetes environment where databases and other StatefulSet applications run, it is crucial to set up alert notifications to continuously monitor and avoid performance degradation, service disruption, downtime or data corruption.

Monitoring tools such as Datadog, Nagios, Prometheus and Grafana can be used to monitor and check database health. They can be integrated with alert notification platforms such as Slack and [PagerDuty](#), so an engineer will receive a notification (often a phone call) whenever there is a degradation in service or another issue with the database.

Conclusion

Running databases in Kubernetes creates unique challenges, including state management, persistent storage and network stability. Administrators can now comfortably manage database workloads in Kubernetes ensuring database integrity and availability by leveraging Kubernetes tools like PersistentVolumes, StorageClasses, StatefulSets and PersistentVolumeClaims.

As Kubernetes continues to evolve, its support for StatefulSets will increase, making running databases in Kubernetes a powerful solution for modern infrastructures.

If you'd like to learn more about what Kubernetes can do for your business, read Andela's article [How to Bridge the DevOps Skills Gap in the Age of Kubernetes and Cloud Native Technologies](#).

TNS



Adetokunbo Ige is a technologist for Andela, a private global talent marketplace. A seasoned platform engineer and a Certified ISO 22301 Lead Implementer in Business Continuity, he brings a wealth of experience in software engineering, enterprise application management, server infrastructure...

Read more from Adetokunbo Ige →

SHARE THIS STORY



TRENDING STORIES

1. The database storage problem is solved. Here's what comes next.
2. When your data model is the bottleneck: lessons from Medium's feature store
3. Why MotherDuck refuses to fork DuckDB
4. Why the need for humans won't disappear in the age of autonomous databases
5. For years, Apache Cassandra handed this work to your team — 6.0 takes it back

INSIGHTS FROM OUR SPONSOR



Andela provides the world's largest private marketplace for global remote tech talent driven by an AI-powered platform to manage the complete contract hiring lifecycle. Andela helps companies scale teams & deliver projects faster via specialized areas: App Engineering, AI, Cloud, Data & Analytics.

[Learn More →](#)

most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)
[Sponsors](#)
[Advertise With Us](#)
[Contributions](#)



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)
[Backend Developer Roadmap](#)
[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)
[Terms of Use](#)
[Advertising Terms & Conditions](#)
[Privacy Policy](#)
[Cookie Policy](#)